

Lab 05 — Multi-stage en images van productiekwaliteit

Theorie — hoe je van een image van 500 MB mét compiler naar een image van 200 MB gaat die alleen bevat wat echt draait; en wat Buildah doet waar Docker BuildKit gebruikt.

Doelstellingen

- Begrijpen waarom je niet dezelfde image gebruikt om te **bouwen** en om te **draaien**.
- Een **multi-stage** build schrijven voor Spring Boot en voor Angular.
- Met kennis van zaken een basisimage kiezen (Debian/Ubuntu, Alpine, distroless).
- Weten wat BuildKit (Docker) en Buildah (Podman) bieden: cache, geheimen, stages.
- Het verband zien tussen imagegrootte, aanvalsoppervlak en uitroltijd.

1. Het probleem: de buildtooling blijft in de image

Een eerste, naïeve Dockerfile voor een Java-API:

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package -DskipTests
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

Hij werkt. Maar hij levert een image van **800 MB tot 1 GB** op, met daarin een volledige JDK (compiler, debugtools), Maven, de lokale repository `~/.m2` met honderden JAR's, je **broncode**, je tests én de uiteindelijke JAR. In productie heb je alleen de JRE en de JAR nodig: zo'n 200 MB.

Dat is meer dan een schoonheidsfoutje:

- **Beveiliging.** Wie de image binnenhaalt, heeft ook je broncode. Elke meegeleverde tool (compiler, `curl`, `git`, shell) is extra gereedschap voor een aanvaller — en een regel meer in het kwetsbaarheidsrapport.
- **Kosten.** 800 MB naar elke node bij elke uitrol, opgeslagen in elke registry, oudere versies inbegrepen.
- **Tijd.** Heeft een node de image nog niet, dan hoort de download bij de start van de container. Bij een incident om 3 uur 's nachts voel je dat verschil.

→ BEVEILIGING

Het **aanvalsoppervlak** van een image is alles wat een aanvaller kan *gebruiken* zodra hij code kan uitvoeren: een shell om rond te kijken, `curl` om data naar buiten te sturen, een compiler, een pakketbeheerder. Elk binary dat ontbreekt, is een hindernis extra voor hem. Daarom tellen scanners (Trivy, Gype) pakketten, en gaat "minimaal" over meer dan megabytes.

2. Multi-stage

Een Dockerfile mag **meerdere FROM-instructies** bevatten. Elke `FROM` opent een *stage*: een onafhankelijke bouwomgeving. Alleen de **laatste** stage wordt de uiteindelijke image; de andere

gooit de engine weg. Met `COPY --from=<stage>` haal je bestanden op uit een eerdere stage.

```
# ----- stage 1: build -----
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ----- stage 2: runtime -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/api.jar app.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

De uiteindelijke image bevat **een JRE en een JAR**. Geen Maven, geen JDK, geen bronnen, geen tests, geen `.m2`. Wat in de stage `build` gebeurde, laat nergens een spoor na — ook niet in verborgen lagen, want die lagen horen simpelweg niet bij de image.

ONTHOUDEN

Multi-stage is ook de enige echt betrouwbare bescherming voor buildgeheimen: wat alleen in een weggegooide stage terechtkwam, bestaat niet in de uiteindelijke image. Let wel op: `COPY --from=build /app /app` zou alles mee kopiëren, geheimen inbegrepen. Kopieer alleen het artefact.

Hetzelfde patroon voor Angular:

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # levert dist/

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist/mijn-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

→ ANGULAR

`ng build` compileert de TypeScript-componenten, bundelt alles tot een handvol `.js` - en `.css` - bestanden plus een `index.html`, minimaliseert ze en zet een vingerafdruk in de bestandsnaam (`main-a1b2c3.js`) voor de browsercache. Het resultaat is **statisch**: elke bestandsserver kan het serveren. `ng serve` is daarentegen een ontwikkelservers die bij elke wijziging hercompileert — onmisbaar op je eigen machine, zinloos in productie.

Eén ding is cruciaal: **Node overleeft de build niet**. Een Angular-frontend in productie is niets anders dan HTML, CSS en JavaScript; een JavaScript-engine aan serverzijde is overbodig. `ng`

`serve` containeriseer je dus **nooit**.

3. Je basisimage kiezen

Basis	Grootte	Voordelen	Nadelen
<code>debian</code> / <code>ubuntu</code>	75-120 MB	Alles werkt, volledige tooling, <code>glibc</code>	Zwaar; veel pakketten, dus veel CVE's
<code>*-slim</code>	30-80 MB	Goed compromis, blijft Debian	Minder tools geïnstalleerd
<code>alpine</code>	5-10 MB	Heel licht, efficiënte <code>apk</code>	Gebruikt musl en niet <code>glibc</code>
<code>distroless</code>	20-50 MB	Geen shell, geen pakketbeheerder	Moeilijk te debuggen, geen <code>exec sh</code>

→ LINUX

De **C-bibliotheek** (`libc`) is de laag tussen programma's en kernel: `printf`, `malloc`, DNS-resolutie, locales. Bijna elk Linux-binary hangt ervan af. `glibc` (GNU) is de historische implementatie, rijk en compatibel; `musl` is een minimalistische herschrijving die Alpine koos omwille van de omvang. Een binary dat voor de ene gecompileerd is, laadt niet met de andere: `ldd --version` in de container vertelt je welke je hebt.

De Alpine-valkuil verdient wat uitleg. De meeste programma's kunnen prima overweg met `musl`, maar niet allemaal: native binaries die voor `glibc` gecompileerd zijn, weigeren te starten; sommige native Java-bibliotheken (compressie, cryptografie, PDF-generatie) falen met een `UnsatisfiedLinkError`; en er duiken verschillen op in DNS-resolutie of locales.

In de praktijk volstaat `eclipse-temurin:21-jre-alpine` voor Spring Boot in verreweg de meeste gevallen, en het halveert de grootte. Breekt een native afhankelijkheid, dan val je terug op `eclipse-temurin:21-jre` (Ubuntu). Zo'n keuze test je — je beslist ze niet op papier.

Distroless images (Google) bevatten alleen de runtime en je applicatie: geen shell, geen `ls`, geen pakketbeheerder. Het aanvalsoppervlak is minimaal, maar `podman exec -it container sh` werkt niet meer — regel je observeerbaarheid dus op voorhand anders.

4. Wat echt weegt

Vier hefbomen, van meest naar minst doeltreffend: (1) **multi-stage** — gooit de buildtooling weg, veruit de grootste winst: 800 MB → 200 MB; (2) **de basisimage** — `-jre` in plaats van `-jdk`, `alpine` in plaats van `ubuntu`; (3) **.dockerignore** — houdt `.git`, `node_modules` en `target` buiten de image; (4) **installatie en opruiming in dezelfde RUN**.

Wat daarentegen **niets** oplevert: bestanden verwijderen in een latere laag. Ze blijven in de image zitten (lab 02). En het aantal lagen op zich maakt voor de grootte nauwelijks iets uit.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman history mijn-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
```

5. BuildKit en Buildah

Docker bouwt met **BuildKit**, Podman met **Buildah**. Beide lezen dezelfde Dockerfile en bieden dezelfde nuttige functies:

- **Ongebruikte stages worden nooit gebouwd.** Een stage die niets bijdraagt aan de uiteindelijke image wordt overgeslagen — Buildah toont `[2/3]` en `[3/3]`, en `[1/3]` verschijnt nergens.
- **--target** stopt de build bij een gekozen stage: `podman build --target build -t api-build .` geeft je de compilatiestage als image, om ze te inspecteren.
- **Persistente caches.** `RUN --mount=type=cache,target=/root/.m2 mvn package` bewaart de Maven-repository **tussen builds door**, zonder ze in de image op te nemen. Op een CI-agent is de tijdswinst enorm.
- **Geheimen.** `RUN --mount=type=secret,id=npnrc ...` maakt een bestand beschikbaar voor de duur van één instructie, zonder het ooit in een laag te schrijven (lab 08).

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests
```

PODMAN

Eén zichtbaar verschil: BuildKit bouwt onafhankelijke stages **parallel**, Buildah één voor één. Nog een: de regel `# syntax=docker/dockerfile:1`, die bij Docker de uitgebreide syntax activeert, wordt door Buildah gewoon **genegeerd** — de `--mount`'s werken er ook zonder. En de cache van `type=cache` leeft in je gebruikersopslag (`~/.local/share/containers/storage`), niet in een daemon: twee gebruikers op dezelfde CI-server hebben elk hun eigen cache.

6. Spring Boot: de lagen van de JAR

Een Spring Boot-JAR weegt 50 MB: 45 MB afhankelijkheden die bijna nooit veranderen, en 5 MB code die bij elke commit verandert. Kopieer je hem in één blok, dan wordt dat één laag van 50 MB die bij elke uitrol volledig opnieuw over het netwerk gaat. Spring Boot kan de JAR zelf opsplitsen:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY target/api.jar api.jar
RUN java -Djarmode=tools -jar api.jar extract --layers --destination extracted

FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/extracted/dependencies/ ./
COPY --from=extract /app/extracted/spring-boot-loader/ ./
COPY --from=extract /app/extracted/snapshot-dependencies/ ./
COPY --from=extract /app/extracted/application/ ./
ENTRYPOINT ["java", "-jar", "app.jar"]
```

De afhankelijkheden vormen een stabiele laag, de code een kleine vluchtige laag: een uitrol verplaatst nog maar enkele MB. Onthoud vooral het **principe**: de laagvolgorde van lab 04, toegepast op de inhoud van een JAR.

7. In het bedrijf

- **Eén Dockerfile per dienst**, multi-stage, mee geversioneerd met de code. De CI heeft Maven noch Node nodig: `podman build` (of `docker build`) volstaat, en zo zijn de CI-build en die op je eigen machine gegarandeerd identiek.
 - **Tests** draaien vaak in een aparte stage (`RUN mvn test`), zodat één rode test de imagebuild doet mislukken.
 - **De kwetsbaarheidsscan** (Trivy, Gype) mikt op de uiteindelijke image. Een minimale image geeft een kort rapport dat iemand ook echt leest — een image van 1 GB geeft 300 CVE's die niemand doorneemt.
 - **De uiteindelijke image draait als niet-root**, op een poort boven 1024, en liefst zonder shell.
-

Onthouden

- Bouwen en draaien horen niet in dezelfde image thuis: daar draait multi-stage om.
- Meerdere `FROM`s = meerdere stages; alleen de laatste wordt de image, en `COPY --from` haalt er het artefact in.
- Node hoort niet thuis in de uiteindelijke image van een Angular-frontend: nginx serveert de statische bestanden.
- Kies `-jre` boven `-jdk`, `alpine` als je native afhankelijkheden het toelaten, en distroless als je zonder shell kunt.
- Alpine gebruikt `musl`, niet `glibc`: valideer dat met een test, nooit uit principe.
- BuildKit en Buildah bieden allebei `--target`, persistente caches en buildgeheimen; Buildah negeert `# syntax=` en bouwt niet parallel. Een bestand verwijderen in een latere laag maakt de image niet kleiner.

Woordenschat

stage: bouwstap die door een `FROM` geopend wordt. — `COPY --from`: bestanden ophalen uit een andere stage of image. — `--target`: de build stoppen bij een gekozen stage. — **distroless**: image zonder shell of pakketbeheerder. — **musl / glibc**: twee implementaties van de C-bibliotheek. — **BuildKit / Buildah**: de build-engines van Docker en Podman. — **cache mount**: cache die builds overleeft, buiten de image. — **aanvalsoppervlak**: de misbruikbare componenten in een image.